

PSEUDOCODE CONVENTIONS

- **Pseudocode** is a **high-level description** of a computer program or algorithm that uses a mix of natural language and simple programming constructs to outline the logic of the solution.
- It is not a programming language with specific syntax but serves as a bridge between human understanding and actual code implementation.
- **Pseudocode conventions** help standardize the way pseudocode is written, making it more readable and universally understood.

Here's an introduction to some common **pseudocode conventions**:

➔ Readability and Consistency

- Use consistent indentation to indicate control structures (e.g., loops and conditionals).
- Be mindful of whitespace and line breaks to improve readability.
- Choose a clear and consistent naming convention for variables and functions.

```
1.      // Good pseudocode formatting
2.      if (condition) {
3.          doSomething();
4.      } else {
5.          doSomethingElse();
6.      }
```

➔ Use of Keywords

©Topperworld

- Employ standardized keywords to denote common programming constructs.
- Examples include “if,” “while,” “for,” “else,” “return,” and “function.”

```
1.      // Using keywords for clarity
2.      if (condition) {
3.          doSomething();
4.      } else {
5.          doSomethingElse();
6.      }
```

➔ Comments and Annotations

- Include comments or annotations to explain complex or non-obvious parts of the pseudocode.
- Comments should provide insight into the logic and purpose of the code.

```
1.      // Calculate the sum of two numbers
2.      sum = number1 + number2;
3.      // Display the result
4.      print(sum);
```

➔ Modularization

- Break down complex algorithms into smaller, manageable functions or procedures.

- Use clear and meaningful function names to describe their purpose.

```
1.     function calculateSum(a, b) {  
2.         return a + b;  
3.     }  
4.  
5.     result = calculateSum(number1, number2);
```

©Topperworld

➔ Use of Control Structures

- Utilize standard control structures such as “if-else,” “while,” and “for” loops to represent conditional and iterative logic.

```
1.     if (condition) {  
2.         // Code to execute if the condition is true  
3.     } else {  
4.         // Code to execute if the condition is false  
5.     }
```

➔ Data Types

- Pseudocode often abstracts data types, but you can use descriptive terms like “integer,” “string,” or “list” when necessary.

```
1.     // Initialize an integer variable  
2.     age = 25;  
3.  
4.     // Create an empty list  
5.     names = [];
```

➔ Error Handling

- If your algorithm involves error handling, use pseudocode to outline how errors are detected and handled.

```
1.      try {  
2.          // Attempt some operation  
3.      } catch (error) {  
4.          // Handle the error  
5.          logError(error);  
6.      }
```

➔ Iteration and Looping

- Clearly define the start and end conditions for loops.
- Indicate how loop variables are updated within the loop.

```
1.      // Loop from 1 to 10  
2.      for i = 1 to 10 {  
3.          // Perform an action for each iteration  
4.          print(i);  
5.      }
```